

NewtonX ADK 使用方法ガイド（NewtonX エージェント開発キット）

目次

1. インストール
2. クイックスタート
3. 認証設定
4. 基本的な使用方法
5. チャット機能
6. ファイルアップロード
 - アシスタントへのナレッジ登録/削除
7. フォルダ管理
8. エラーハンドリング
9. ベストプラクティス
10. トラブルシューティング
11. セットアップツールと設定の確認

インストール

pipを使用したインストール

```
pip install newtonx-adk
```

開発版のインストール（ご利用には事前申請が必要です）

```
git clone https://github.com/innovationserakucojpororganization/newtonx_adk.git
cd newtonx-python-adk
pip install -e .
```

クイックスタート

1. 基本的なセットアップ

```
from newtonx_adk import NewtonXClient, ConfigManager

# 設定管理クラスを初期化
config_manager = ConfigManager()

# Host と PAT を設定（または事前にセットアップツールで登録済み）
config_manager.update_config(
    host="your-subdomain.newton-x.net",
    personal_access_token="your_pat_token",
```

```
)

# クライアントを初期化 (PAT があれば追加の認証は不要)
client = NewtonXClient(config_manager)

print("セットアップ完了")
```

2. 簡単なチャット例

```
# アシスタント一覧を取得
assistants = client.get_assistants()

# チャットを作成
chat_uid = client.create_chat(
    assistant_uid=assistants[0]['uid'],
    title="テストチャット"
)

# メッセージを送信
response = client.send_message(chat_uid, "こんにちは!")
print(f"アシスタントの応答: {response}")
```

認証設定

PAT 認証 (Host + PAT)

NewtonX ADK は Personal Access Token (PAT) による認証を使用します。NewtonX の Web 版で発行した PAT と、利用中のホスト名 (例: [seraku.newton-x.net](#)) を設定すれば動作します。

1. PAT の登録 (推奨: セットアップツール)

```
python tools/setup_config.py
```

- 入力項目
 - Company Subdomain (例: [seraku](#)) → [seraku.newton-x.net](#) を自動設定
 - Personal Access Token (PAT)

2. コードから直接設定する場合

```
from newtonx_adk import ConfigManager

cm = ConfigManager()
cm.update_config(
    host="seraku.newton-x.net",
```

```
personal_access_token="your_pat_token",  
)
```

3. 認証の確認（ヘッダー検証）

```
from newtonx_adk import AuthManager, ConfigManager  
  
cm = ConfigManager()  
am = AuthManager(cm)  
headers = am.get_headers()  
assert headers.get("Authorization", "").startswith("Bearer ")
```

基本的な使用方法

アシスタントの利用

```
# アシスタント一覧を取得  
assistants = client.get_assistants()  
  
# アシスタント情報を表示  
for assistant in assistants:  
    print(f"名前: {assistant['name']}")  
    print(f"ID: {assistant['uid']}")  
    print(f"説明: {assistant.get('description', 'N/A')}")  
    print("----")  
  
# 特定のアシスタントを選択  
selected_assistant = assistants[0] # 最初のアシスタント
```

チャットの作成と管理

```
# チャットを作成  
chat_uid = client.create_chat(  
    assistant_uid=selected_assistant['uid'],  
    title="新しいチャット"  
)  
  
# チャット詳細を取得  
chat_detail = client.get_chat(chat_uid)  
print(f"チャットタイトル: {chat_detail['title']}")  
print(f"アシスタント: {chat_detail['assistant_name']}")  
  
# チャットタイトルを更新  
client.update_chat_title(chat_uid, "更新されたタイトル")
```

```
# チャットを削除
client.delete_chat(chat_uid)
```

チャット一覧の取得

```
# 全チャットを取得
chats = client.get_chats()

# 検索クエリでフィルタリング
search_results = client.get_chats(search_query="会議")

# ページネーション
page2_chats = client.get_chats(page=2, page_size=10)

# チャット情報を表示
for chat in chats:
    print(f"タイトル: {chat['title']}")
    print(f"ID: {chat['uid']}")
    print(f"作成日: {chat.get('created_at', 'N/A')}")
    print("----")
```

チャット機能

メッセージの送信

```
# 基本的なメッセージ送信
response = client.send_message(
    chat_uid=chat_uid,
    message="こんにちは！"
)

# 検索設定を指定
response = client.send_message(
    chat_uid=chat_uid,
    message="最新のAI技術について教えてください",
    web_search=True,          # ウェブ検索を有効
    knowledge_search=False   # ナレッジ検索を無効
)

# 画像付きメッセージ
image_id = client.upload_image(chat_uid, "image.jpg")
response = client.send_message(
    chat_uid=chat_uid,
    message="この画像を分析してください",
    image_ids=[image_id]
)
```

複数画像をまとめて送って解析する

```
# 画像をまとめてアップロードしてからメッセージ送信する場合
image_ids = client.upload_images(chat_uid, ["/path/to/a.png", "/path/to/b.jpg"])
# ["uidA", "uidB"]
response = client.send_message(
    chat_uid=chat_uid,
    message="これらの画像を比較して要点を抽出してください",
    image_ids=image_ids,
)

# 画像アップロードと送信を一括で行うヘルパー
response = client.send_message_with_images(
    chat_uid=chat_uid,
    message="これらの画像から読み取れる内容を教えて",
    image_file_paths=["/path/to/a.png", "/path/to/b.jpg"],
)
```

音声付きメッセージの送信

音声ファイルを添付してメッセージを送信できます。音声ファイルはメッセージと一緒に送信され、AIアシスタントが音声の内容を理解して応答します。

```
# 音声ファイルを添付してメッセージを送信
response = client.send_message(
    chat_uid=chat_uid,
    message="この音声を要約してください",
    audio_file_path="/path/to/sample.wav"
)

if response:
    print(f"アシスタントの応答: {response}")
```

対応ファイル形式と制限

- 対応ファイル形式: .wav, .mp3, .aiff, .aac, .ogg, .flac
- 最大ファイルサイズ: 15MB

注意事項

- `audio_file_path` を指定した場合、メッセージは `multipart/form-data` 形式で送信されます
- `image_ids` と `audio_file_path` は同時に指定できません（相互排他）
- 音声ファイルが存在しない場合は `FileUploadError` が発生します
- ファイルサイズが15MBを超える場合は、アップロード前にエラーが発生する可能性があります

```
# エラーハンドリングの例
from newtonx_adk import FileUploadError, APIError
```

```
try:
    response = client.send_message(
        chat_uid=chat_uid,
        message="この音声を文字起こししてください",
        audio_file_path="/path/to/audio.mp3"
    )
except FileUploadError as e:
    print(f"ファイルアップロードエラー: {e}")
except APIError as e:
    print(f"APIエラー: {e}")
```

メッセージの応答処理

```
# メッセージを送信して応答を処理
response = client.send_message(chat_uid, "質問内容")

if response:
    print(f"アシスタントの応答: {response}")

    # 応答をファイルに保存
    with open("response.txt", "w", encoding="utf-8") as f:
        f.write(response)
else:
    print("メッセージの送信に失敗しました")
```

ファイルアップロード

画像ファイルのアップロード

```
# 画像ファイルをアップロード
image_id = client.upload_image(
    chat_uid=chat_uid,
    file_path="path/to/image.jpg",
    file_name="custom_name.jpg" # オプション
)

if image_id:
    print(f"画像がアップロードされました: {image_id}")

    # 画像付きメッセージを送信
    response = client.send_message(
        chat_uid=chat_uid,
        message="この画像を分析してください",
        image_ids=[image_id]
    )
else:
    print("画像のアップロードに失敗しました")
```

ドキュメントファイルのアップロード

```
# ドキュメントファイルをアップロード
success = client.upload_document(
    chat_uid=chat_uid,
    file_path="path/to/document.pdf",
    file_name="custom_name.pdf" # オプション
)

if success:
    print("ドキュメントがアップロードされました")

    # ドキュメントについて質問
    response = client.send_message(
        chat_uid=chat_uid,
        message="このドキュメントの内容を要約してください"
    )
else:
    print("ドキュメントのアップロードに失敗しました")
```

サポートされているファイル形式

画像ファイル

- JPEG (.jpg, .jpeg)
- PNG (.png)
- GIF (.gif)

ドキュメントファイル

- PDF (.pdf)
- Word (.doc, .docx)
- Excel (.xls, .xlsx)
- PowerPoint (.ppt, .pptx)
- テキスト (.txt)

音声ファイル

- WAV (.wav)
- MP3 (.mp3)
- AIFF (.aiff)
- AAC (.aac)
- OGG (.ogg)
- FLAC (.flac)

注意: 音声ファイルは `send_message` の `audio_file_path` パラメータで直接送信します。最大ファイルサイズは15MBです。

フォルダ管理

フォルダの作成と管理

```
# フォルダを作成
folder_uid = client.create_folder("新しいフォルダ")

if folder_uid:
    print(f"フォルダが作成されました: {folder_uid}")

    # チャットをフォルダに移動
    success = client.move_chat_to_folder(chat_uid, folder_uid)
    if success:
        print("チャットがフォルダに移動されました")
else:
    print("フォルダの作成に失敗しました")
```

アシスタントへのナレッジ登録/削除

アシスタントに紐づくRAG（ナレッジ）としてファイルを登録・削除できます。対応形式は `.docx`, `.xls`, `.xlsx`, `.pptx`, `.pdf`, `.txt` で、16MB以下のファイルに制限されます。

```
# アシスタントを取得
assistants = client.get_assistants()
assistant_uid = assistants[0]['uid']

# ナレッジを登録
file_uid = client.add_assistant_knowledge(
    assistant_uid=assistant_uid,
    file_path="docs/company_policy.pdf",
)

# 登録したナレッジを削除（IDが取得できない環境ではNoneとなる場合があります）
if file_uid:
    ok = client.delete_assistant_knowledge(assistant_uid, file_uid)
    print("削除成功" if ok else "削除失敗")
```

フォルダー一覧の取得


```
# フォルダ一覧を取得
folders = client.get_folders()

for folder in folders:
    print(f"フォルダ名: {folder['name']}")
    print(f"フォルダID: {folder['uid']}")
    print(f"作成日: {folder.get('created_at', 'N/A')}")
    print("----")
```

フォルダ内のチャット一覧

```
# フォルダ内のチャット一覧を取得
folder_chats = client.get_folder_chats(folder_uid)

for chat in folder_chats:
    print(f"チャットタイトル: {chat['title']}")
    print(f"チャットID: {chat['uid']}")
    print("----")
```

エラーハンドリング

例外の種類

```
from newtonx_adk import NewtonXError, AuthenticationError, APIError

try:
    response = client.send_message(chat_uid, "テストメッセージ")
except AuthenticationError as e:
    print(f"認証エラー: {e}")
    # 認証を再実行
    client.authenticate()
except APIError as e:
    print(f"APIエラー: {e}")
    print(f"ステータスコード: {e.status_code}")
    print(f"レスポンス: {e.response}")
except NewtonXError as e:
    print(f"その他のエラー: {e}")
except Exception as e:
    print(f"予期しないエラー: {e}")
```

リトライ機能

```
import time
from newtonx_adk import APIError
```

```
def send_message_with_retry(client, chat_uid, message, max_retries=3):
    """リトライ機能付きメッセージ送信"""
    for attempt in range(max_retries):
        try:
            return client.send_message(chat_uid, message)
        except APIError as e:
            if attempt < max_retries - 1:
                print(f"APIエラー、{attempt + 1}回目のリトライ: {e}")
                time.sleep(2 ** attempt) # 指数バックオフ
            else:
                raise e

# 使用例
try:
    response = send_message_with_retry(client, chat_uid, "テストメッセージ")
    print(f"応答: {response}")
except Exception as e:
    print(f"最終的に失敗: {e}")
```

ベストプラクティス

1. 設定の管理

```
# 設定を一元管理
config_manager = ConfigManager("my_config.json")

# 環境に応じて設定を変更
if os.getenv("ENVIRONMENT") == "production":
    config_manager.update_config(
        api_base_url="https://api.newtonx.com",
        timeout=60
    )
else:
    config_manager.update_config(
        api_base_url="https://staging-api.newtonx.com",
        timeout=30
    )
```

2. 認証の管理

```
# 認証状態を定期的にチェック
def ensure_authenticated(client):
    """認証状態を確保"""
    if not client.auth_manager.is_authenticated():
        print("認証が必要です")
        if not client.authenticate():
            raise Exception("認証に失敗しました")
    return True
```

```
# 使用例
ensure_authenticated(client)
response = client.send_message(chat_uid, "メッセージ")
```

3. エラーハンドリング

```
def safe_api_call(func, *args, **kwargs):
    """安全なAPI呼び出し"""
    try:
        return func(*args, **kwargs)
    except AuthenticationError:
        print("認証エラー、再認証を試行")
        client.authenticate()
        return func(*args, **kwargs)
    except APIError as e:
        print(f"APIエラー: {e}")
        return None
    except Exception as e:
        print(f"予期しないエラー: {e}")
        return None

# 使用例
response = safe_api_call(client.send_message, chat_uid, "メッセージ")
```

4. リソースの管理

```
class NewtonXManager:
    """NewtonXリソース管理クラス"""

    def __init__(self):
        self.config_manager = ConfigManager()
        self.client = NewtonXClient(self.config_manager)
        self.chats = []
        self.folders = []

    def create_chat(self, assistant_uid, title):
        """チャットを作成して管理"""
        chat_uid = self.client.create_chat(assistant_uid, title)
        if chat_uid:
            self.chats.append(chat_uid)
        return chat_uid

    def cleanup(self):
        """リソースのクリーンアップ"""
        for chat_uid in self.chats:
            try:
                self.client.delete_chat(chat_uid)
            except Exception as e:
                print(f"チャット削除エラー {chat_uid}: {e}")
```

```
# 使用例
manager = NewtonXManager()
try:
    chat_uid = manager.create_chat(assistant_uid, "テストチャット")
    response = manager.client.send_message(chat_uid, "メッセージ")
finally:
    manager.cleanup()
```

トラブルシューティング

よくある問題と解決方法

1. 認証エラー

問題: `AuthenticationError`: 認証が必要です または `Authorization` ヘッダーが付与されない

解決方法:

```
# 対話的セットアップで Host と PAT を再登録
python tools/setup_config.py

# 設定の確認 (Authorization が Bearer で始まること)
python tools/check_config.py
```

セットアップツールと設定の確認

概要

Host と PAT (Personal Access Token) を登録・確認するためのツール群です。

使用例 (Host-first 推奨)

```
# 対話的セットアップ (Company Subdomain と PAT を入力)
python tools/setup_config.py

# 設定の表示/確認 (JSON出力、Authorization などを確認)
python tools/check_config.py
```

Host-first について

サブドメイン (例: `seraku`) を入力すると、以下が自動で設定されます。

- `host`: `seraku.newton-x.net`
- `api_base_url`: `https://seraku.newton-x.net/api`
- `company_subdomain`: `seraku`

明示的に上書きも可能です (`ConfigManager.update_config(...)`)。

2. API通信エラー

問題: `APIError`: API通信エラー

解決方法:

```
# ネットワーク接続を確認
import requests
try:
    response = requests.get("https://api.newtonx.com", timeout=5)
    print("ネットワーク接続: OK")
except Exception as e:
    print(f"ネットワーク接続エラー: {e}")

# タイムアウト設定を調整
config_manager.update_config(timeout=60)
```

3. ファイルアップロードエラー

問題: `FileUploadError`: ファイルが存在しません

解決方法:

```
import os

# ファイルの存在確認
file_path = "path/to/file.jpg"
if os.path.exists(file_path):
    print("ファイルが存在します")
    # ファイルサイズを確認
    file_size = os.path.getsize(file_path)
    print(f"ファイルサイズ: {file_size} bytes")
else:
    print("ファイルが存在しません")
```

4. メモリ不足エラー

問題: 大きなファイルのアップロード時にメモリ不足

解決方法:

```
# ファイルを分割してアップロード
def upload_large_file(client, chat_uid, file_path, chunk_size=1024*1024):
    """大きなファイルを分割アップロード"""
    with open(file_path, 'rb') as f:
        chunk_num = 0
```

```
while True:
    chunk = f.read(chunk_size)
    if not chunk:
        break

    # チャンクを一時ファイルに保存
    temp_file = f"temp_chunk_{chunk_num}.tmp"
    with open(temp_file, 'wb') as temp:
        temp.write(chunk)

    # チャンクをアップロード
    try:
        client.upload_document(chat_uid, temp_file)
        print(f"チャンク {chunk_num} をアップロードしました")
    finally:
        os.remove(temp_file)

    chunk_num += 1
```

デバッグ情報の取得

```
import logging

# デバッグログを有効化
logging.basicConfig(level=logging.DEBUG)

# ADKのデバッグ情報を取得
print(f"ADK Version: {newtonx_adk.__version__}")
print(f"Config: {config_manager.get_config()}")
print(f"Authenticated: {client.auth_manager.is_authenticated()}")
```

パフォーマンスの最適化

```
# 並列処理で複数のチャットを処理
import concurrent.futures

def process_chat(chat_uid, message):
    """チャットを処理"""
    return client.send_message(chat_uid, message)

# 並列実行
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    futures = []
    for chat_uid in chat_uids:
        future = executor.submit(process_chat, chat_uid, "メッセージ")
        futures.append(future)

# 結果を収集
for future in concurrent.futures.as_completed(futures):
```

```
try:
    response = future.result()
    print(f"応答: {response}")
except Exception as e:
    print(f"エラー: {e}")
```